

Thinking: 60% 的准确率够用吗?

如果每个 Head 只选 Top-1 做验证, 就退化为 Blockwise Parallel Decoding (每步最多接受 1 个额外 token)。

60% 意味着经常猜错。所以 MEDUSA 不选 Top-1, 而是选 Top-k, 构建树状候选结构 -- 这就是 Tree Attention。



图中红框的部分, 它是 Head 3 的预测 token 吗, 为什么只有 2 个, 为什么只有一个 Head 2 预测 token 有 Head 3 的预测 token ?

<https://gemini.google.com/app/21d39e366127ce06>

你观察得非常敏锐! 这正是 MEDUSA 在工程实现中为了“性价比”而做出的一个非常精妙的架构设计。

我们来逐一解答你的这三个疑问, 这涉及到 MEDUSA 的“稀疏树结构 (Sparse Tree Structure)”设计:

1. 它是 Head 3 的预测 token 吗?

是的, 完全正确。

根据图中的层级关系:

- 根节点: 原始 LM Head (预测第 1 个词)
- 第一层 (A, B): Medusa Head 1 的输出 (预测第 2 个词)
- 第二层 (A-1 到 B-3): Medusa Head 2 的输出 (预测第 3 个词)
- 第三层 (红框 A-1-a, A-1-b): 正是 Medusa Head 3 的输出 (预测第 4 个词)。

2. 为什么只有 2 个?

这是因为在配置这棵“草稿树”的拓扑结构时, 人为或系统设定了 Head 3 在这个分支上只取 Top-2 的预测结果。

这与预测的**难度 (不确定性)**有关。大模型往后盲猜越远, 准确率就越低。猜第 4 个词比猜第 2 个词难得多, 所以系统不需要让 Head 3 给出太多备选项 (给多了也是错的, 纯属浪费算力), 取概率最高的 2 个就足够了。

3. 为什么只有一个 Head 2 (A-1) 下面挂了 Head 3?

这是 MEDUSA 最核心的算力优化策略——避免指数级爆炸, 把算力集中在“刀刃”上。

你可以想象一下，如果这棵树是“完全对称”的展开(全排列组合)：

- Head 1 取 2 个候选
- Head 2 取 3 个候选
- Head 3 取 2 个候选

那么总共的路径会变成： $2 \times 3 \times 2 = 12$ 条路径！如果后面还有 Head 4、Head 5，候选路径会呈指数级爆炸，导致 GPU 单次前向传播的 Tree Attention Mask 矩阵变得极其巨大，反而拖慢了速度。

因此，**MEDUSA** 采用了不对称的“稀疏树”：

系统知道，候选 **A** 的概率通常大于候选 **B**；而在候选 **A** 下面，**A-1** 的概率肯定远大于 **A-2** 和 **A-3**。既然 **A-1** 是当前所有走势中**“最有可能发生(概率最高)”的一条主线路径，那么系统就只把 Head 3 的预测资源(算力)挂在 **A-1** 下面**，去继续往下深挖。

对于像 **A-2**、**A-3** 或者 **B-1** 这种本来概率就偏低的“冷门路径”，系统认为它们最终被大模型接受的希望不大，就不再浪费 Head 3 的算力去为它们预测后续走向了，直接“剪枝”停止展开。

💡 总结：

图中的红框确实是 Head 3 的预测。之所以它只出现在 **A-1** 下面且只有 2 个，是因为 **MEDUSA** 在构建草稿树时，故意砍掉了低概率分支的后续展开，将有限的并行算力(候选位)全部倾斜分配给了概率最高的那条主线路径，以此在“命中率”和“计算开销”之间达到完美的平衡。